

Exercise Sheet 1, 2026

6.0 VU Advanced Database Systems

SERHII SLUTU (12537831)

March 20, 2026

Exercise 1 (Functions and Stored procedures) [*3 Punkte*]

1. Create a function that computes the total sales for all months as follows.

```
CREATE OR REPLACE FUNCTION get_monthly_sales(p_product_id INT, p_year INT)
RETURNS TABLE(month_num INT, total_sales NUMERIC)
AS $$
DECLARE
    i INT;
    monthly_total NUMERIC;
BEGIN
    FOR i IN 1..12 LOOP
        SELECT COALESCE(SUM(quantity * unit_sale_price), 0)
        INTO monthly_total
        FROM go_daily_sales
        WHERE product_number = p_product_id
            AND EXTRACT(YEAR FROM date) = p_year
            AND EXTRACT(MONTH FROM date) = i;

        month_num := i;
        total_sales := monthly_total;
        RETURN NEXT;
    END LOOP;
END;
$$ LANGUAGE plpgsql;

//example
SELECT * FROM get_monthly_sales(109110, 2015);
```

2. Rewrite the function using a cursor to improve its efficiency.

```
CREATE OR REPLACE FUNCTION get_monthly_sales_cursor(
    p_product_id INT, p_year INT)
RETURNS TABLE(month_num INT, total_sales NUMERIC)
AS $$
```

```

DECLARE
    i INT;
    db_month INT;
    db_total NUMERIC;

    sales_cursor CURSOR FOR
        SELECT
            EXTRACT(MONTH FROM date)::INT AS month_num,
            SUM(quantity * unit_sale_price) AS total_sales
        FROM go_daily_sales
        WHERE product_number = p_product_id
            AND EXTRACT(YEAR FROM date) = p_year
        GROUP BY EXTRACT(MONTH FROM date)
        ORDER BY EXTRACT(MONTH FROM date);
BEGIN
    OPEN sales_cursor;

    FETCH sales_cursor INTO db_month, db_total;

    FOR i IN 1..12 LOOP
        IF db_month = i THEN
            month_num := i;
            total_sales := COALESCE(db_total, 0);
            RETURN NEXT;

            FETCH sales_cursor INTO db_month, db_total;
        ELSE
            month_num := i;
            total_sales := 0;
            RETURN NEXT;
        END IF;
    END LOOP;

    CLOSE sales_cursor;
END;
$$ LANGUAGE plpgsql;

SELECT * FROM get_monthly_sales_cursor(109110, 2015);

```

3. Discuss the difference in efficiency of the two approaches.

Since cursor allows us to perform 1 heavy aggregation instead of 12 in this case it outperforms the initial function almost by 20ms.

Exercise 2 (Triggers) [2 Punkte]

1. CREATE OR REPLACE FUNCTION validate_last_sale_date()

```
RETURNS TRIGGER
AS $$
BEGIN
    IF OLD.last_sale_date IS NOT NULL
        AND NEW.last_sale_date < OLD.last_sale_date THEN
        RAISE EXCEPTION
            'last_sale_date cannot move backwards: old=%, new=%',
            OLD.last_sale_date, NEW.last_sale_date;
    END IF;

    RAISE NOTICE
        'last_sale_date updated successfully for product %: % -> %',
        NEW.product_number, OLD.last_sale_date, NEW.last_sale_date;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_validate_last_sale_date
BEFORE UPDATE OF last_sale_date ON go_products
FOR EACH ROW
EXECUTE FUNCTION validate_last_sale_date();
```

2. CREATE OR REPLACE FUNCTION update_last_sale_date_after_insert()

```
RETURNS TRIGGER
AS $$
BEGIN
    RAISE NOTICE 'Statement-level trigger start';

    UPDATE go_products
    SET last_sale_date = CURRENT_DATE
    WHERE product_number IN (
        SELECT DISTINCT product_number
        FROM new_sales
    );

    RAISE NOTICE 'Statement-level trigger end';

    RETURN NULL;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_update_last_sale_date_after_insert
AFTER INSERT ON go_daily_sales
REFERENCING NEW TABLE AS new_sales
FOR EACH STATEMENT
EXECUTE FUNCTION update_last_sale_date_after_insert();
```

Test A (valid update):

```
UPDATE go_products
SET last_sale_date = DATE '2026-03-01'
WHERE product_number = 109110;
```

Output:

```
NOTICE: last_sale_date updated successfully for product 109110:
        <NULL> -> 2026-03-01
```

Test B (invalid update):

```
UPDATE go_products
SET last_sale_date = DATE '2026-02-01'
WHERE product_number = 109110;
```

Output:

```
ERROR: last_sale_date cannot move backwards:
        old=2026-03-20, new=2026-02-01
```

This confirms that the row-level trigger correctly prevents updates where the new date is earlier than the existing one.

Test C (prepare single insert):

```
UPDATE go_products
SET last_sale_date = DATE '2026-03-10'
WHERE product_number = 109110;
```

Test D (single INSERT):

```
INSERT INTO go_daily_sales
        (retailer_code, product_number, order_method_code,
         date, quantity, unit_price, unit_sale_price)
VALUES
        (1201, 109110, 4, CURRENT_DATE, 5, 100, 90);
```

Output:

```
NOTICE: Statement-level trigger start
NOTICE: last_sale_date updated successfully for product 109110:
        2026-03-10 -> 2026-03-20
NOTICE: Statement-level trigger end
```

The statement-level trigger fires once for the INSERT. It performs an UPDATE on go_products, which causes the row-level trigger to fire once for the affected product.

Test E (prepare bulk insert):

```
UPDATE go_products
SET last_sale_date = DATE '2026-03-10'
WHERE product_number IN (112110, 115110);
```

Test F (bulk INSERT):

```
INSERT INTO go_daily_sales
        (retailer_code, product_number, order_method_code,
         date, quantity, unit_price, unit_sale_price)
VALUES
```

```
(1201, 112110, 4, CURRENT_DATE, 2, 100, 90),
(1201, 115110, 4, CURRENT_DATE, 4, 200, 180),
(1201, 112110, 4, CURRENT_DATE, 1, 100, 95);
```

Output:

```
NOTICE: Statement-level trigger start
NOTICE: last_sale_date updated successfully for product 112110:
        2026-03-10 -> 2026-03-20
NOTICE: last_sale_date updated successfully for product 115110:
        2026-03-10 -> 2026-03-20
NOTICE: Statement-level trigger end
```

For the bulk INSERT, the statement-level trigger still fires only once because it is defined with `FOR EACH STATEMENT`. However, the row-level trigger fires once per updated row in `go_products`. In this case, it fired for two products.

Execution order:

- INSERT on `go_daily_sales`
- statement-level trigger fires once
- UPDATE on `go_products`
- row-level trigger fires once per updated row

Exercise 3 (Datalog Basics) [*3 Punkte*] We consider the flight graph

```
flight("Vienna", "Berlin").
flight("Vienna", "Paris").
flight("Berlin", "Amsterdam").
flight("Paris", "Amsterdam").
flight("Paris", "Barcelona").
flight("Amsterdam", "Barcelona").
flight("Barcelona", "Porto").
flight("Amsterdam", "Porto").
```

and the following non-linear Datalog programme:

```
route(?x, ?y) :- flight(?x, ?y).
route(?x, ?y) :- route(?x, ?z), route(?z, ?y).
```

Since the recursive rule contains two occurrences of the IDB predicate `route`, the programme is non-linear. Therefore we use the optimised semi-naive evaluation:

Initialization:

```
Delta^1_route(?x, ?y) :- flight(?x, ?y).
route^1(?x, ?y) :- Delta^1_route(?x, ?y).
```

Loop:

```
temp^(i+1)_route(?x,?y) :- Delta^i_route(?x,?z), route^(i-1)(?z,?y).
temp^(i+1)_route(?x,?y) :- route^i(?x,?z), Delta^i_route(?z,?y).
Delta^(i+1)_route := temp^(i+1)_route - route^i
route^(i+1) := route^i union Delta^(i+1)_route
```

Initialization

$$\Delta_{route}^1 = \{(\text{Vienna}, \text{Berlin}), (\text{Vienna}, \text{Paris}), (\text{Berlin}, \text{Amsterdam}), (\text{Paris}, \text{Amsterdam}), \\ (\text{Paris}, \text{Barcelona}), (\text{Amsterdam}, \text{Barcelona}), (\text{Barcelona}, \text{Porto}), (\text{Amsterdam}, \text{Porto})\}$$

$$route^1 = \Delta_{route}^1$$

Step 2

Using the two semi-naive rules, we obtain:

$$temp_{route}^2 = \{(\text{Vienna}, \text{Amsterdam}), (\text{Vienna}, \text{Barcelona}), (\text{Berlin}, \text{Barcelona}), \\ (\text{Berlin}, \text{Porto}), (\text{Paris}, \text{Barcelona}), (\text{Paris}, \text{Porto}), (\text{Amsterdam}, \text{Porto})\}$$

Now remove tuples already contained in $route^1$:

$$\Delta_{route}^2 = temp_{route}^2 - route^1 \\ = \{(\text{Vienna}, \text{Amsterdam}), (\text{Vienna}, \text{Barcelona}), \\ (\text{Berlin}, \text{Barcelona}), (\text{Berlin}, \text{Porto}), (\text{Paris}, \text{Porto})\}$$

$$route^2 = route^1 \cup \Delta_{route}^2$$

Hence

$$route^2 = \{(\text{Vienna}, \text{Berlin}), (\text{Vienna}, \text{Paris}), (\text{Berlin}, \text{Amsterdam}), \\ (\text{Paris}, \text{Amsterdam}), (\text{Paris}, \text{Barcelona}), \\ (\text{Amsterdam}, \text{Barcelona}), (\text{Barcelona}, \text{Porto}), (\text{Amsterdam}, \text{Porto}), \\ (\text{Vienna}, \text{Amsterdam}), (\text{Vienna}, \text{Barcelona}), \\ (\text{Berlin}, \text{Barcelona}), (\text{Berlin}, \text{Porto}), (\text{Paris}, \text{Porto})\}$$

Step 3

Using Δ_{route}^2 , we derive:

$$temp_{route}^3 = \{(\text{Vienna}, \text{Barcelona}), (\text{Vienna}, \text{Porto}), (\text{Berlin}, \text{Porto})\}$$

Again, remove tuples already known in $route^2$:

$$\Delta_{route}^3 = temp_{route}^3 - route^2 \\ = \{(\text{Vienna}, \text{Porto})\}$$

$$route^3 = route^2 \cup \Delta_{route}^3$$

$$route^3 = \{(\text{Vienna}, \text{Berlin}), (\text{Vienna}, \text{Paris}), (\text{Berlin}, \text{Amsterdam}), \\ (\text{Paris}, \text{Amsterdam}), (\text{Paris}, \text{Barcelona}), \\ (\text{Amsterdam}, \text{Barcelona}), (\text{Barcelona}, \text{Porto}), (\text{Amsterdam}, \text{Porto}), \\ (\text{Vienna}, \text{Amsterdam}), (\text{Vienna}, \text{Barcelona}), \\ (\text{Berlin}, \text{Barcelona}), (\text{Berlin}, \text{Porto}), (\text{Paris}, \text{Porto}), (\text{Vienna}, \text{Porto})\}$$

Step 4

Since Porto has no outgoing flight, no further new tuples can be derived:

$$temp_{route}^4 = \emptyset$$

$$\Delta_{route}^4 = \emptyset$$

$$route^4 = route^3$$

Therefore the least fixpoint is reached in Step 4.

Discussion

The auxiliary relation $temp_{route}^{i+1}$ contains all tuples that can be derived by new rule firings in round $i+1$, but some of these tuples may already be known from earlier rounds. Therefore Δ_{route}^{i+1} is obtained by subtracting the already known relation $route^i$. The fixpoint is reached as soon as $\Delta_{route}^{i+1} = \emptyset$.

Exercise 4 (Datalog with Negation) [3 Punkte]

```
% Employee Database Dataset
% Data imported via CSV resources
@import employee :- csv{
    resource = "https://dbai.tuwien.ac.at/adbs/datalog/data/employees/employees.csv",
    format=(int,string,string,string,int,int,int)} .
@import department :- csv{
    resource = "https://dbai.tuwien.ac.at/adbs/datalog/data/employees/departments.csv",
    format=(int,string,int,string)} .

% === Add your rules below ===

% ManagersOfAtLeastOneEmployee = pi_manager_id(employee)
manages_at_least_one_employee(?manager_id) :-
    employee(?employee_id, ?first_name, ?last_name,
        ?job_id, ?salary, ?manager_id, ?department_id).

% DepartmentHeads = pi_manager_id(department)
is_department_head(?manager_id) :-
    department(?department_id, ?department_name,
        ?manager_id, ?location).

% ManagersWhoAreNotDepartmentHeads = S - DeptHeads
manages_but_is_not_department_head(?manager_id) :-
    manages_at_least_one_employee(?manager_id),
    ~is_department_head(?manager_id).

% Full employee tuples of managers who are not department heads
eligible_manager_employee(?employee_id, ?first_name, ?last_name,
    ?job_id, ?salary, ?manager_id, ?department_id) :-
    employee(?employee_id, ?first_name, ?last_name,
        ?job_id, ?salary, ?manager_id, ?department_id),
    manages_but_is_not_department_head(?employee_id).

% Executive department IDs
executive_department(?department_id) :-
```

```

department(?department_id, ?department_name,
           ?department_manager_id, ?location),
?department_name = "Executive".

% Managers who manage at least one employee in the Executive department
manages_employee_in_executive(?manager_id) :-
    employee(?employee_id, ?first_name, ?last_name,
             ?job_id, ?salary, ?manager_id, ?department_id),
    executive_department(?department_id).

% Candidate managers that must be excluded
excluded_manager(?employee_id, ?first_name, ?last_name,
                 ?job_id, ?salary, ?manager_id, ?department_id) :-
    eligible_manager_employee(?employee_id, ?first_name, ?last_name,
                              ?job_id, ?salary, ?manager_id, ?department_id),
    manages_employee_in_executive(?employee_id).

% Final result
result(?employee_id, ?first_name, ?last_name,
       ?job_id, ?salary, ?manager_id, ?department_id) :-
    eligible_manager_employee(?employee_id, ?first_name, ?last_name,
                              ?job_id, ?salary, ?manager_id, ?department_id),
    ~excluded_manager(?employee_id, ?first_name, ?last_name,
                      ?job_id, ?salary, ?manager_id, ?department_id).

@output result .

```

Stratification and Evaluation

The program is stratified based on predicate dependencies.

Stratum 1: employee, department, manages_at_least_one_employee, is_department_head, executive_department

Stratum 2: manages_but_is_not_department_head, eligible_manager_employee, manages_employee, excluded_manager

Stratum 3: result

Negation requires a strictly higher stratum. Therefore, `manages_but_is_not_department_head` depends negatively on `is_department_head` (Stratum 1), and `result` depends negatively on `excluded_manager` (Stratum 2).

Evaluation proceeds bottom-up: first Stratum 1 is computed from base relations, then Stratum 2 using these results, and finally Stratum 3.

Exercise 5 (CTEs) [2 Punkte]

1. Create a materialized view that aggregates the monthly sales per product:

```

CREATE MATERIALIZED VIEW monthly_product_sales AS
SELECT
    product_number,
    EXTRACT(YEAR FROM date) AS year,
    EXTRACT(MONTH FROM date) AS month,
    SUM(quantity * unit_sale_price) AS total_revenue
FROM go_daily_sales
GROUP BY product_number, year, month;

```

2. Use a CTE to find the top-selling month per product for the year 2018:

```
WITH ranked_sales AS (
  SELECT
    product_number,
    month,
    total_revenue,
    RANK() OVER (
      PARTITION BY product_number
      ORDER BY total_revenue DESC
    ) AS rank
  FROM monthly_product_sales
  WHERE year = 2018
)
SELECT
  product_number,
  month,
  total_revenue
FROM ranked_sales
WHERE rank = 1;
```

Exercise 6 (CTEs with Recursion) [3 Punkte]

1. WITH RECURSIVE distinct_sales_dates AS (
- ```
 SELECT DISTINCT
 product_number,
 date
 FROM go_daily_sales
),
streaks AS (
 SELECT
 product_number,
 date AS start_date,
 date AS current_date,
 1 AS streak_length
 FROM distinct_sales_dates

 UNION ALL

 SELECT
 s.product_number,
 s.start_date,
 d.date AS current_date,
 s.streak_length + 1
 FROM streaks s
 JOIN distinct_sales_dates d
 ON d.product_number = s.product_number
 AND d.date = s.current_date + INTERVAL '1 day'
),
```

```

max_streaks AS (
 SELECT
 product_number,
 MAX(streak_length) AS longest_streak
 FROM streaks
 GROUP BY product_number
)
SELECT
 product_number,
 longest_streak
FROM max_streaks
ORDER BY longest_streak DESC, product_number;

2. CREATE TEMP TABLE temp_distinct_sales_dates AS
SELECT DISTINCT
 product_number,
 date
FROM go_daily_sales;

WITH RECURSIVE streaks AS (
 SELECT
 product_number,
 date AS start_date,
 date AS current_date,
 1 AS streak_length
 FROM temp_distinct_sales_dates

 UNION ALL

 SELECT
 s.product_number,
 s.start_date,
 d.date AS current_date,
 s.streak_length + 1
 FROM streaks s
 JOIN temp_distinct_sales_dates d
 ON d.product_number = s.product_number
 AND d.date = s.current_date + INTERVAL '1 day'
),
max_streaks AS (
 SELECT
 product_number,
 MAX(streak_length) AS longest_streak
 FROM streaks
 GROUP BY product_number
)
SELECT
 product_number,
 longest_streak

```

```
FROM max_streaks
ORDER BY longest_streak DESC, product_number;
```